



**UNIVERSIDAD
NACIONAL AUTÓNOMA
DE MÉXICO**



**FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN**

Sistemas Distribuidos

Mexi Banco con Compose · sábado 19 de septiembre (S5)

NOMBRE DE LA ALUMNA: Muñoz San Agustin Victoria Monserrat

NOMBRE DEL PROFESOR: Oscar Manuel Ruiz Hurtado

GRUPO: 02

SEMESTRE 2027-1

FECHA DE ENTREGA: Sábado 27 de septiembre de 2026

Para la realización de este laboratorio fue necesaria la instalación de Postman



PASO 1: Clona el repo

Una vez instalada debíamos clonar el repo en nuestra máquina para poder trabajar con los elementos.

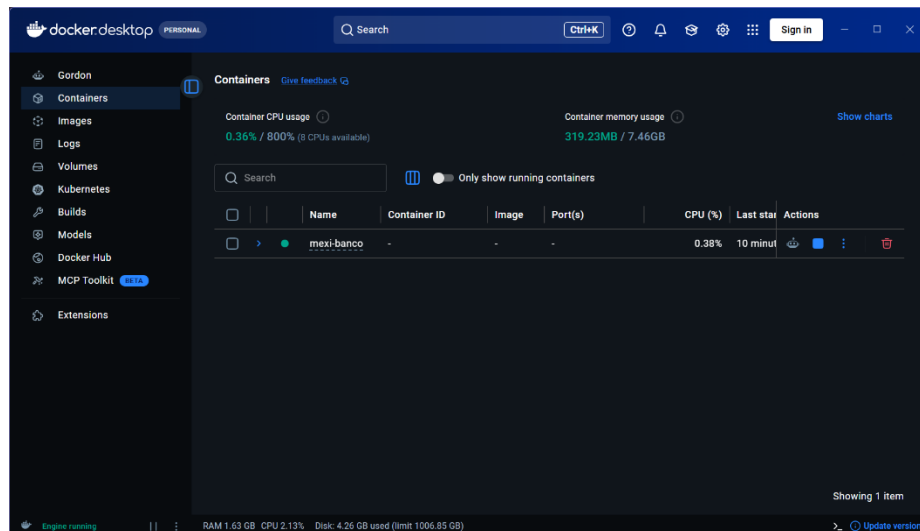
```
victo@brannvico MINGW64 ~  
$ git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git  
cd mexi-banco  
Cloning into 'mexi-banco'...  
remote: Enumerating objects: 123, done.  
remote: Counting objects: 100% (123/123), done.  
remote: Compressing objects: 100% (97/97), done.  
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)  
Receiving objects: 100% (123/123), 46.71 KiB | 713.00 KiB/s, done.  
Resolving deltas: 100% (23/23), done.  
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!  
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.  
  
You are in 'detached HEAD' state. You can look around, make experimental  
changes and commit them, and you can discard any commits you make in this  
state without impacting any branches by switching back to a branch.  
  
If you want to create a new branch to retain commits you create, you may  
do so (now or later) by using -c with the switch command. Example:  
  
    git switch -c <new-branch-name>  
  
Or undo this operation with:  
  
    git switch -  
  
Turn off this advice by setting config variable advice.detachedHead to false
```

IMPORTANTE: también no debía olvidarse descargar la colección (lo resalto porque fue un paso que al no tomarlo en cuenta me perdí en el repo buscando el archivo JSON)

 **Descargar la colección de Postman**

PASO 2: Levántalo con Compose

Para poder orquestar y levantar de forma automatizada toda la infraestructura local de Mexi Banco con un solo comando primero debe tenerse a Docker corriendo, si no dará error al mandar el comando en la terminal



Una vez con el Docker levantado mande el comando, tardo alrededor de 5 min y mostraba una gran cantidad de texto porque la bandera --build obliga a reconstruir la imagen de la aplicación desde cero, lo que implica descargar las imágenes base necesarias, ejecutar todo el proceso de compilación y empaquetado del código fuente mediante Maven a través de múltiples etapas, y desplegar la inicialización interna del motor de base de datos hasta que los contenedores logran enlazarse de forma totalmente operativa.

```
victo@brannvici MINGW64 ~/mexi-banco ((v05.1))
$ docker compose up --build -d
docker compose ps
#1 [internal] load local bake definitions
#1 reading from stdin 503B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 440B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#3 DONE 0.5s

#4 [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
#4 DONE 0.5s
```

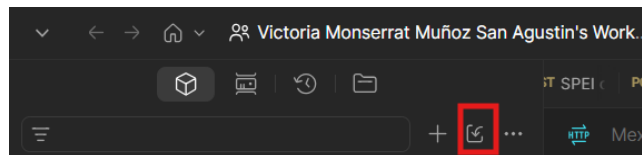
```
#19 resolving provenance for metadata file
#19 DONE 0.0s
[+] up 5/5
✔ Image mexi-banco-app Built 60.8s
✔ Network mexibanco Created 0.0s
✔ Volume mexi-banco_mexibanco_datos Created 0.0s
✔ Container mexi-banco-db-1 Healthy 6.1s
✔ Container mexi-banco-app-1 Started 6.3s
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
mexi-banco-app-1 mexi-banco-app "java -jar app.jar" app 8 seconds ago Up 1 second (health: starting) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1 postgres:17-alpine "docker-entrypoint.s..." db 8 seconds ago Up 7 seco
```

Una vez cargado lo anterior se ejecutó el siguiente comando para confirmar que los contenedores estaban activos y operando.

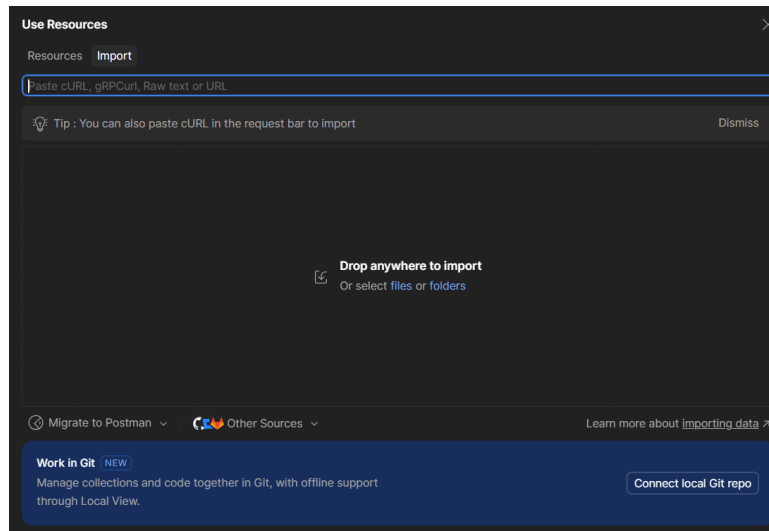
```
victo@brannvict MINGW64 ~/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        9 minutes ago  Up 9 minu    8080/tcp
tes (healthy)
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s... db         9 minutes ago  Up 9 minu    5432/tcp
tes (healthy)
```

PASO 3: Prueba los endpoints

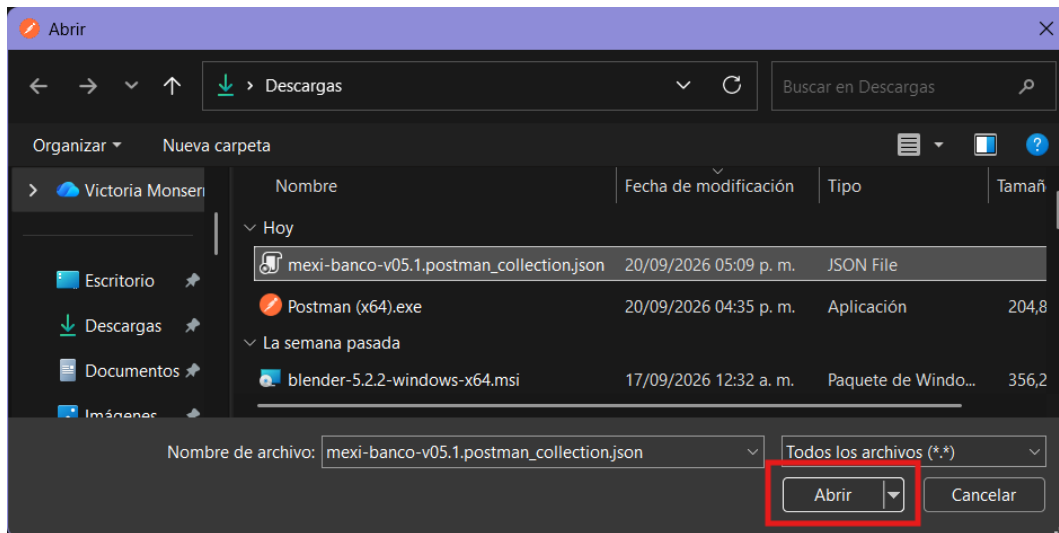
Después de los comandos debíamos iniciar el paso 3, el cual consistía en hacer uso de la colección descargada en el paso 1. Para esto primero debí ver un video en YouTube porque la interfaz en Postman era nueva para mi y no encontraba los botones mencionados en la práctica. Dentro de Postman del lado superior izquierdo se encontraba el icono siguiente:



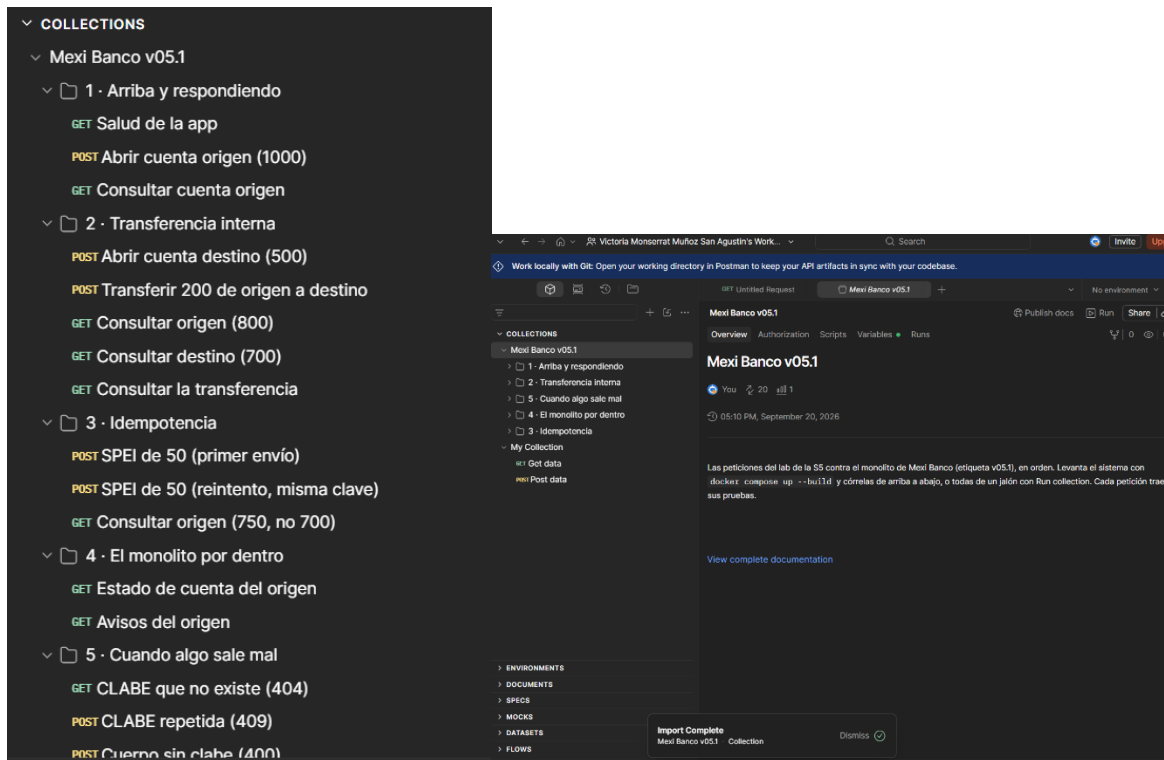
Luego arroja esta pestaña



Yo seleccione “files” y me direcciono a otra pestaña que configure así:



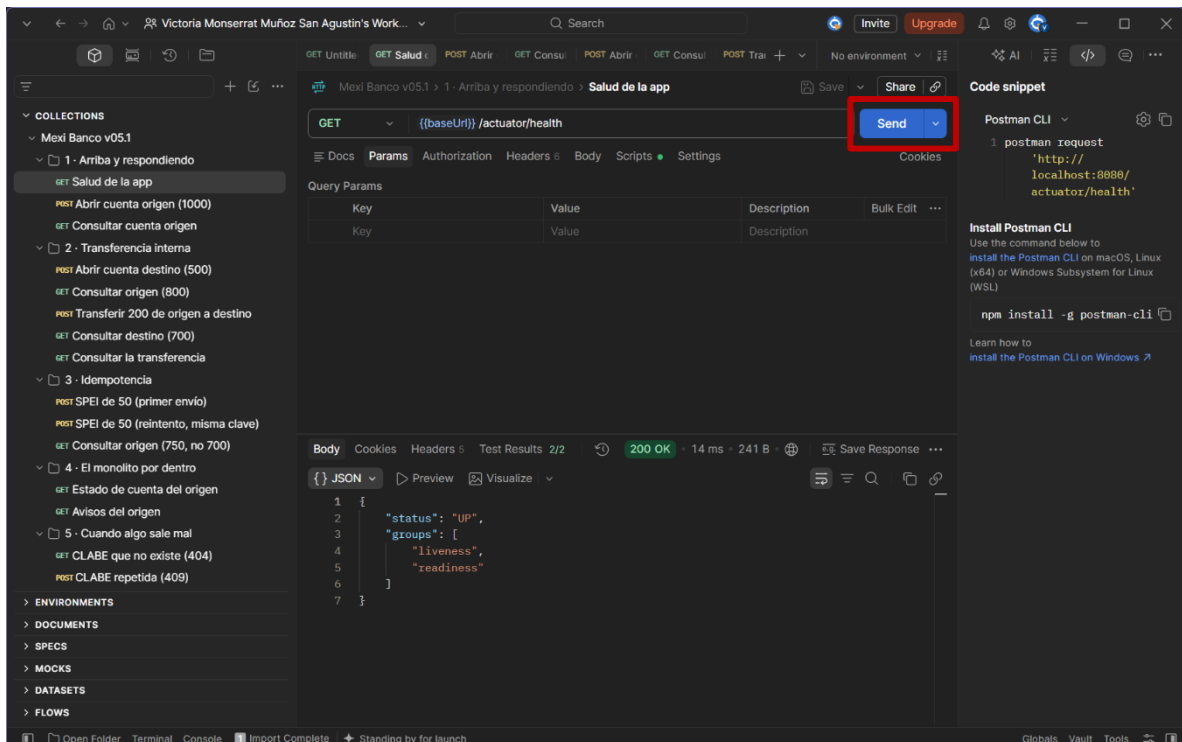
Al abrirlos Postman te cargara todos los métodos necesarios



Cada uno se debe enviar para recibir las respectivas respuestas de cada método

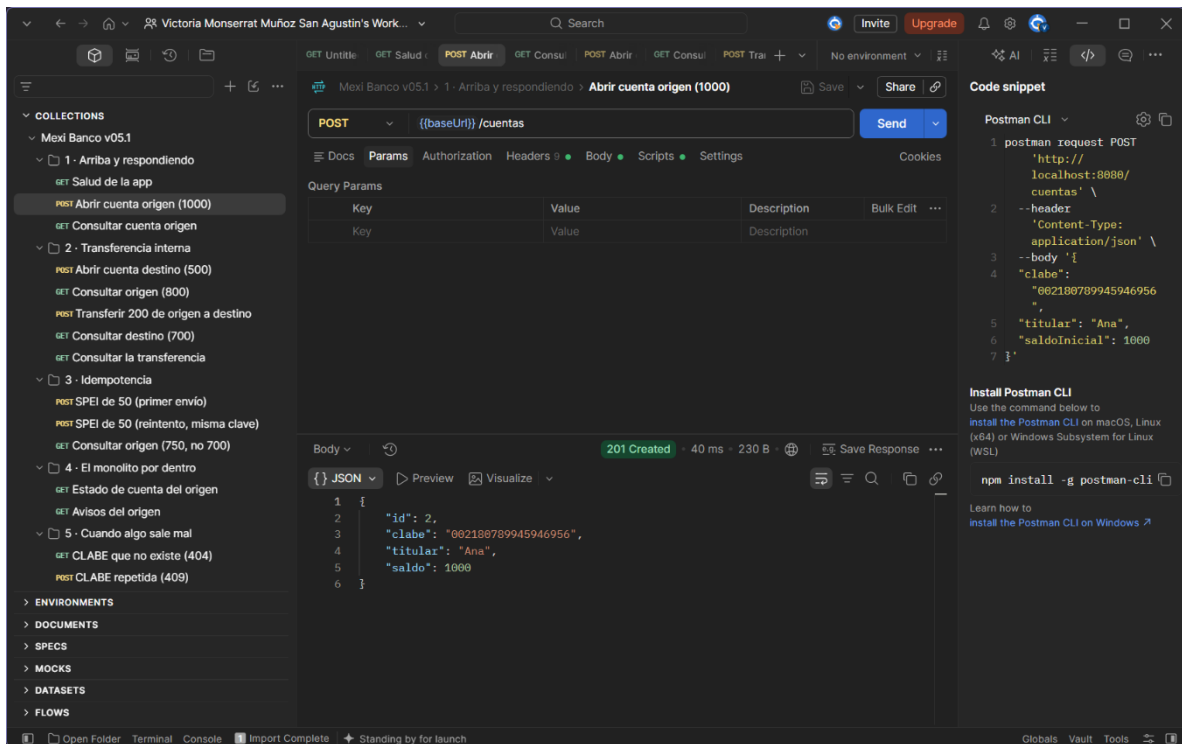
1. Arriba y respondiendo

1.1. Salud de la app: `{{baseUrl}}/actuador/health`



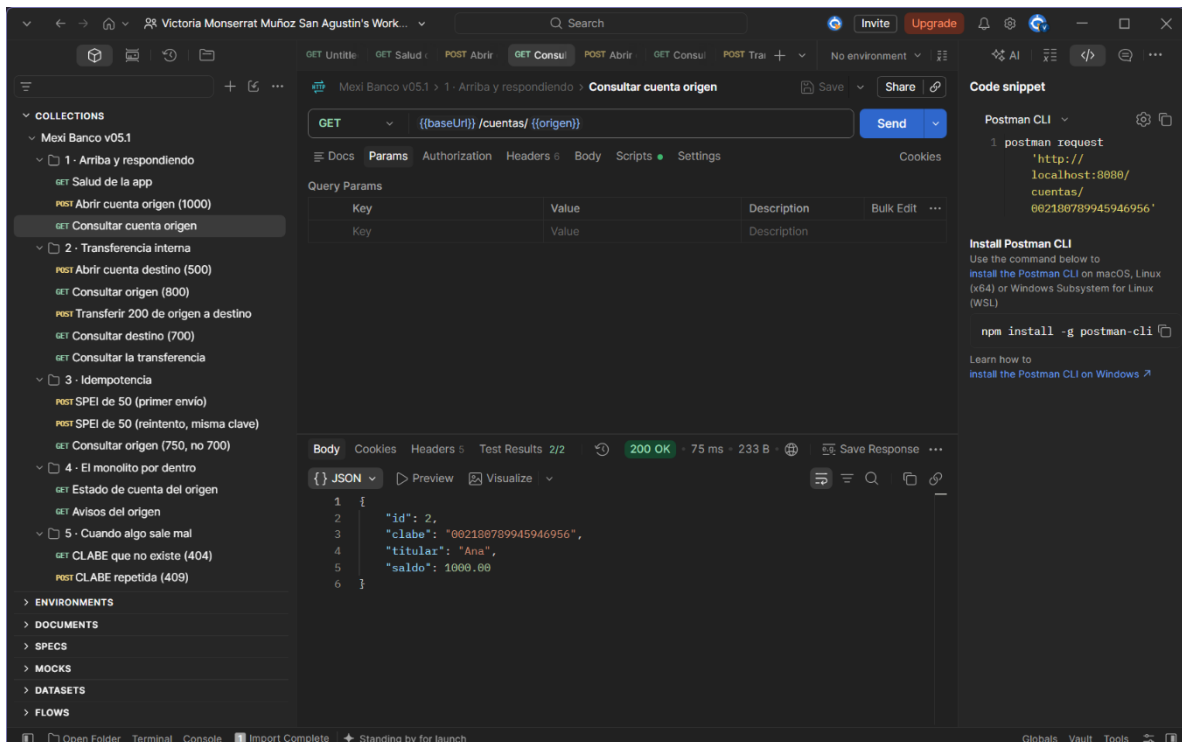
Se realiza una solicitud `GET` al endpoint de diagnóstico, donde la respuesta confirma de manera gráfica y exitosa que el monolito se encuentra operando y respondiendo correctamente en el entorno local.

1.2. Abrir cuenta origen: `{{baseUrl}}/cuentas`



Se observa la ejecución de la petición `POST` para abrir una nueva cuenta dentro de la colección de Mexi Banco. En el cuerpo de la respuesta en formato JSON se aprecian los detalles del registro creado, incluyendo el identificador interno, la CLABE bancaria asignada, el nombre de la titular ("Ana") y el saldo inicial configurado de 1000.

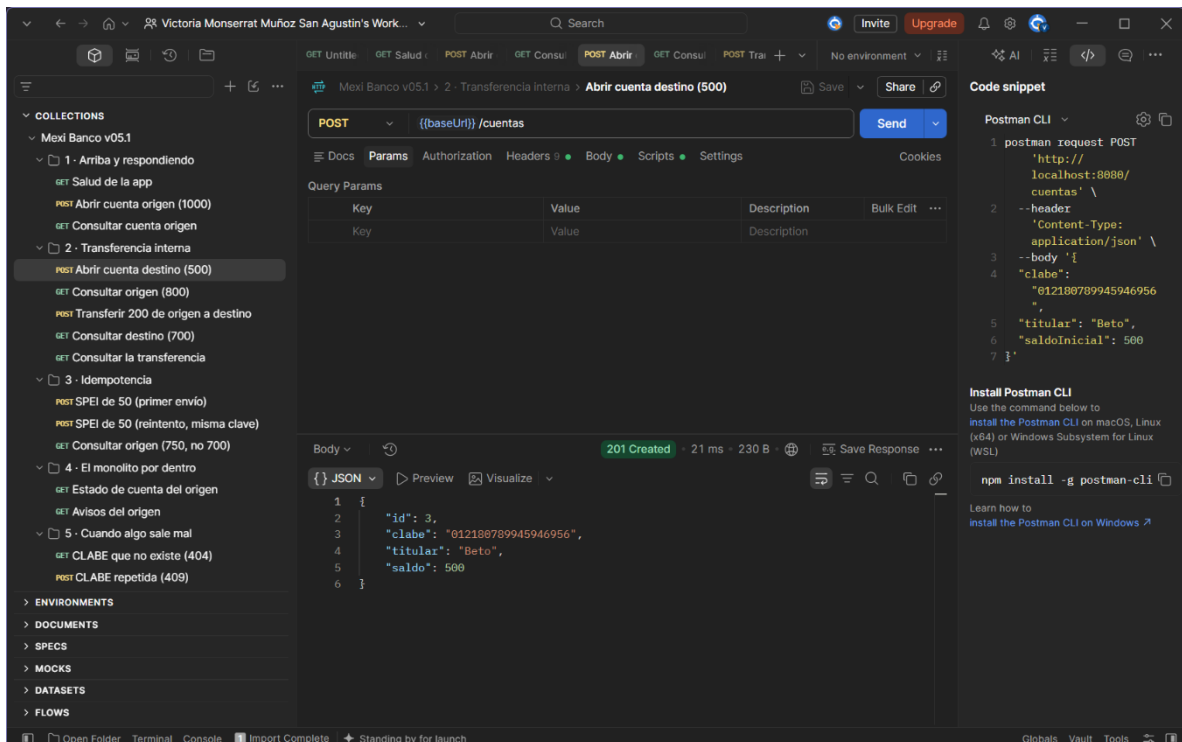
1.3. Consultar cuenta origen: `{{baseUrl}}/cuentas/{{origen}}`



Aquí es una petición `GET` para consultar los datos de la cuenta de origen, la cual despliega la información detallada de la cuenta previamente registrada.

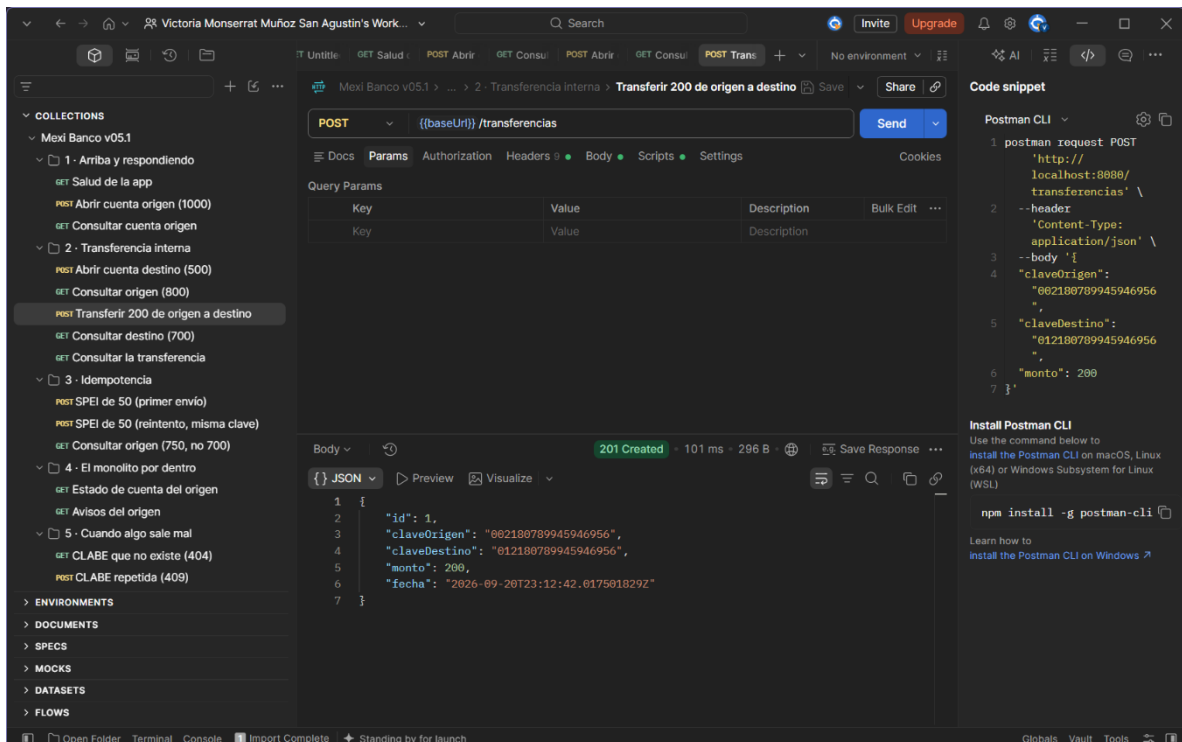
2. Transferencia interna

2.1. Abrir cuenta destino: `{{baseUrl}}/cuentas`



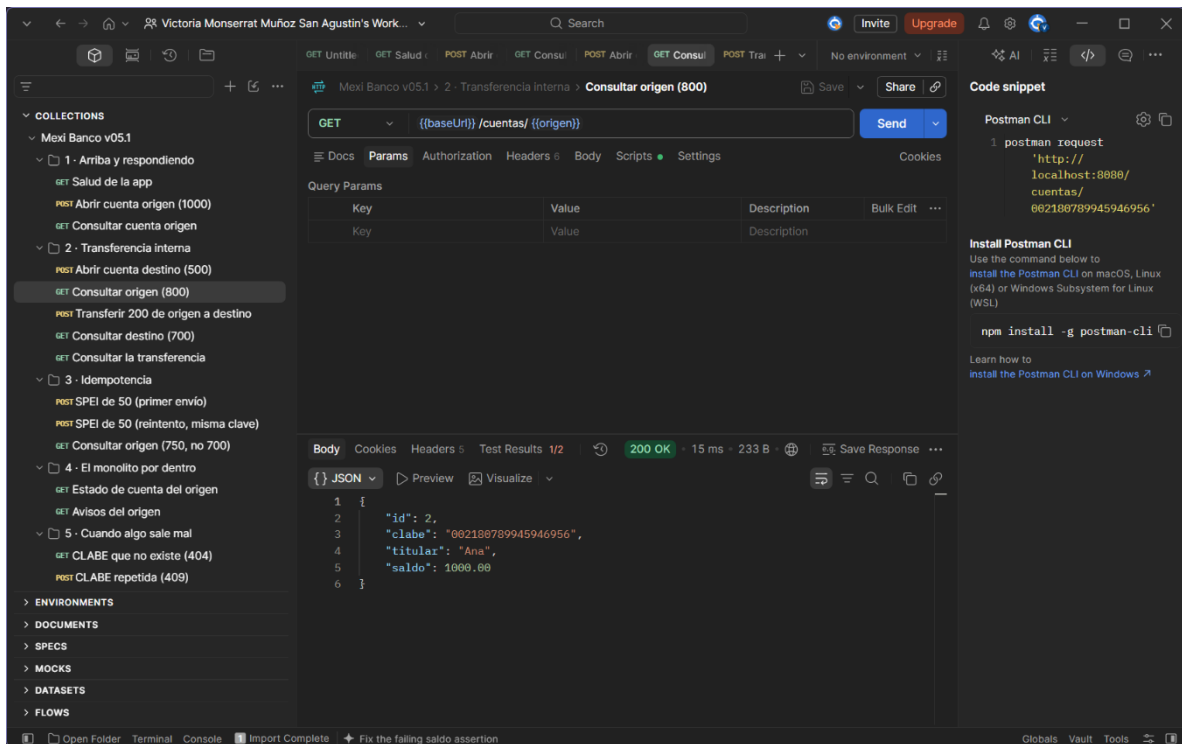
Es una petición `POST` para abrir una cuenta de destino, donde se detallan los datos del nuevo registro creado, asignando un identificador, una nueva CLABE bancaria, el nombre del titular ("Beto") y un saldo inicial de 500.

2.2. Transferir 200 de origen a destino: {{baseUrl}}/transferencias



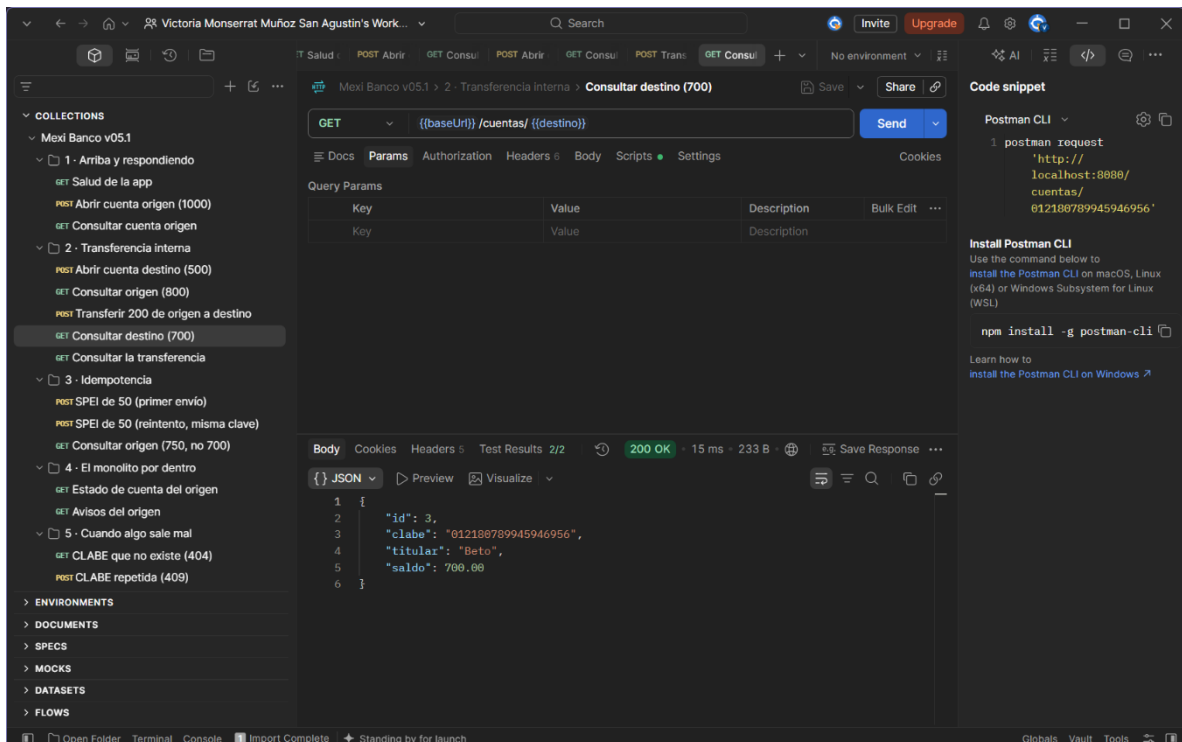
Otra 'POST' correspondiente a la transferencia interna de fondos por un monto de 200 entre las cuentas previamente creadas dentro de la colección, la cual detalla los datos registrados de la transacción.

2.3. Consultar origen: {{baseUrl}}/cuentas/{{origen}}



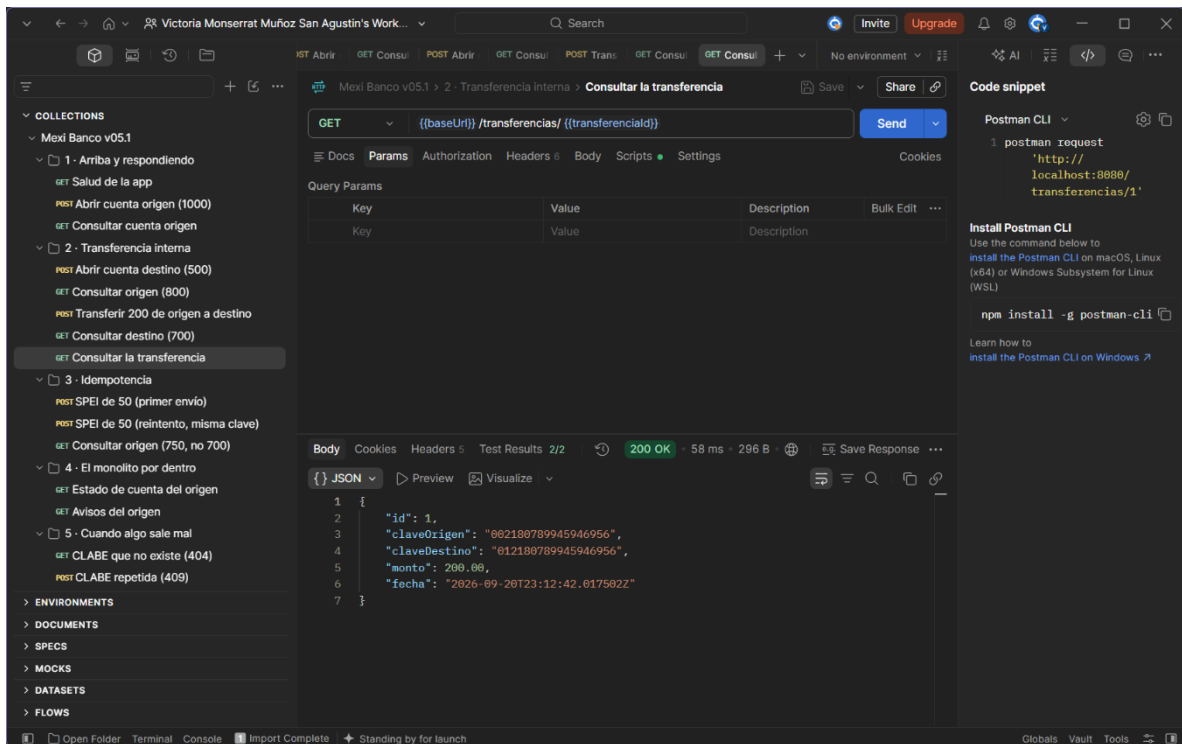
Es `GET` para consultar los datos de la cuenta de origen después de la transferencia. En la respuesta se aprecia que el saldo de la titular "Ana" se mantiene en 1000.00, lo cual indica que en esta prueba los fondos aún no reflejan el descuento de la transferencia.

2.4. Consultar destino: `{{baseUrl}}/cuentas/{{destino}}`



Con la petición `GET` se consulta los datos de la cuenta de destino donde se ve se la cuenta del titular "Beto" refleja un saldo actualizado de `700.00`, lo cual evidencia de manera exitosa la transferencia.

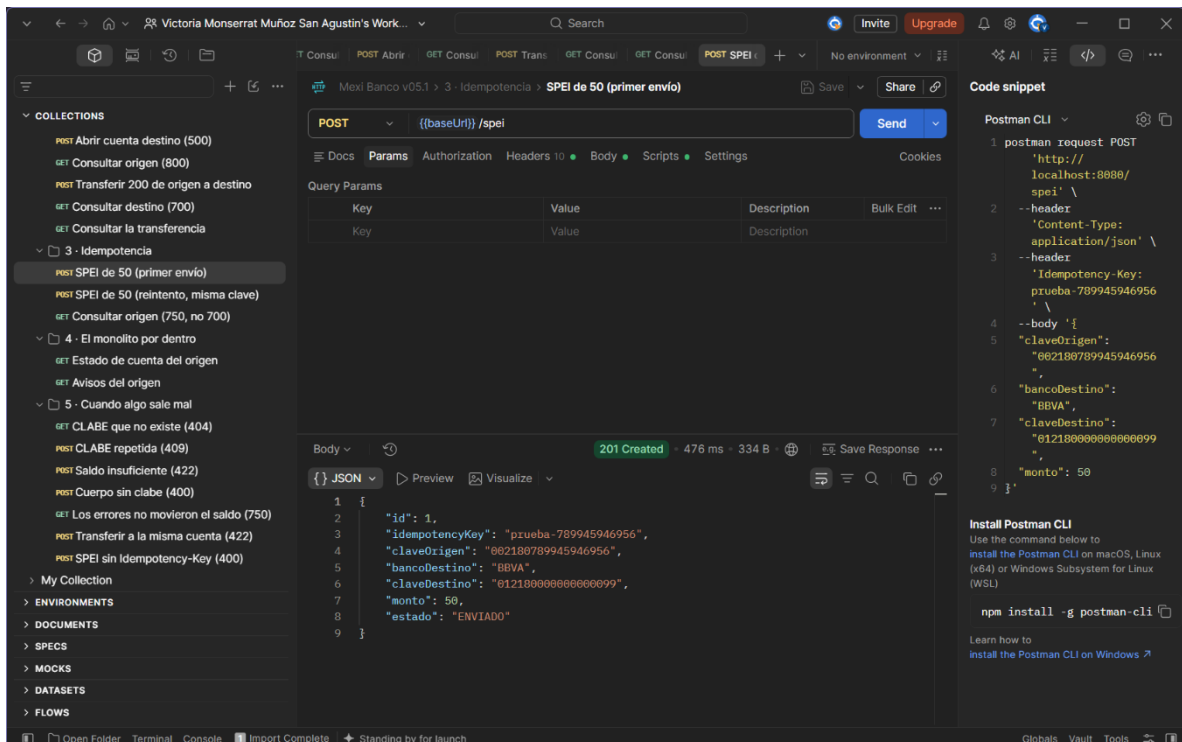
2.5. Consultar la transferencia: `{{baseUrl}}/transferencias/{{transferenciaId}}`



`GET` para la consulta de la transferencia realizada donde se detallan los datos específicos de la operación registrada con el identificador 1.

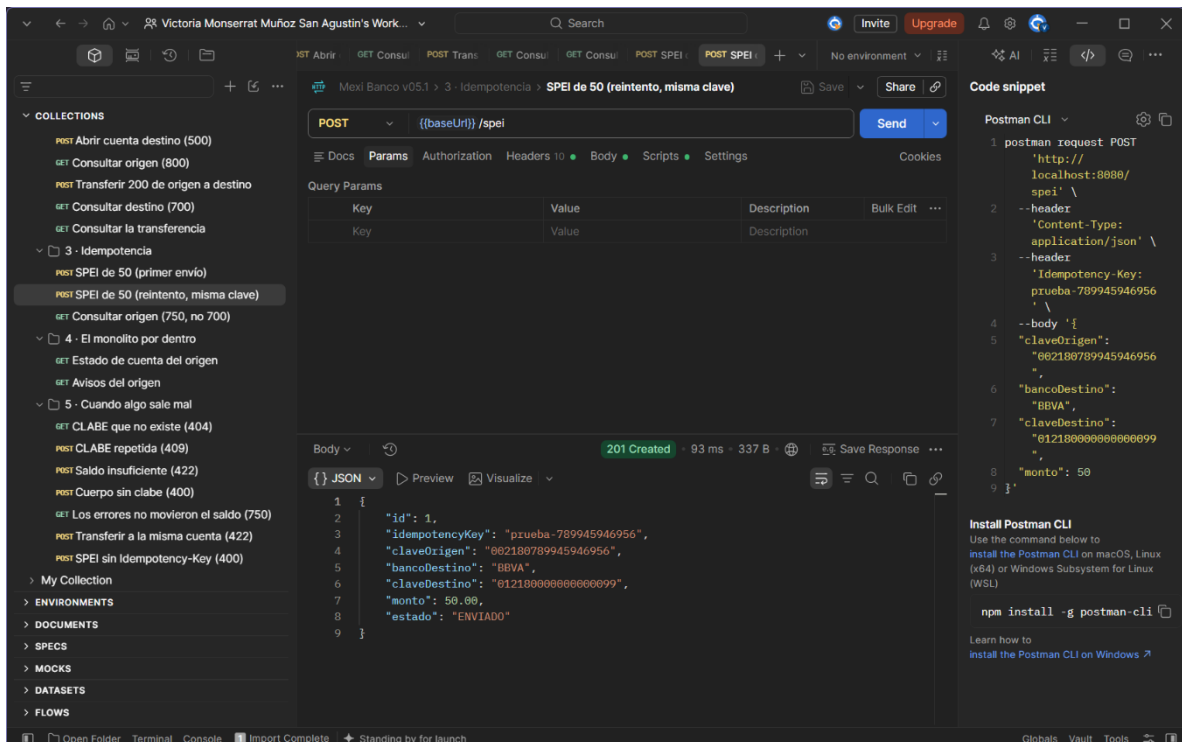
3. Idempotencia

3.1. SPEI de 50 (primer envío): `{{baseUrl}}/spei`



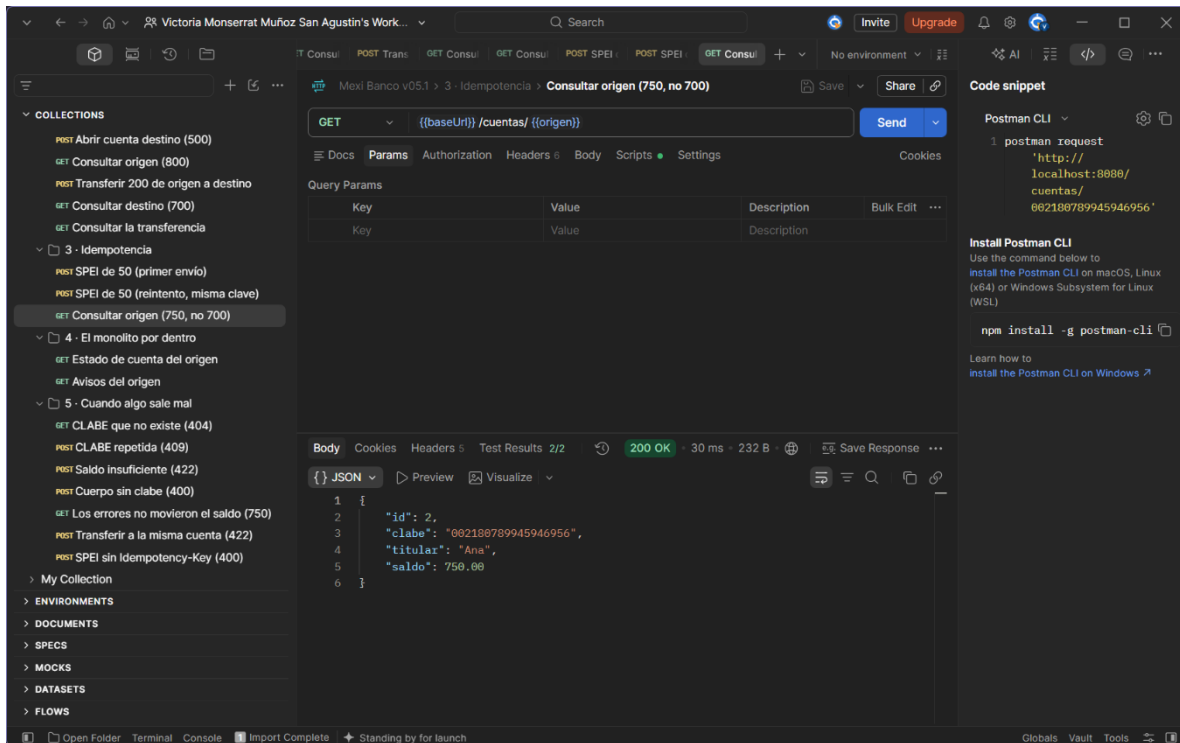
`POST` para el primer envío de una transferencia SPEI. Donde se visualiza `Idempotency-Key` junto con los datos enviados en el cuerpo

3.2. SPEI de 50 (reintento, misma clave): {{baseUrl}}/spei



`POST` correspondiente al reintento de la transferencia SPEI utilizando la misma clave de idempotencia, el sistema devuelve exactamente la misma información de la transacción original registrada con el identificador 1 y el estado "ENVIADO", validando de manera exitosa que la lógica de idempotencia evita procesar o cobrar la operación por duplicado.

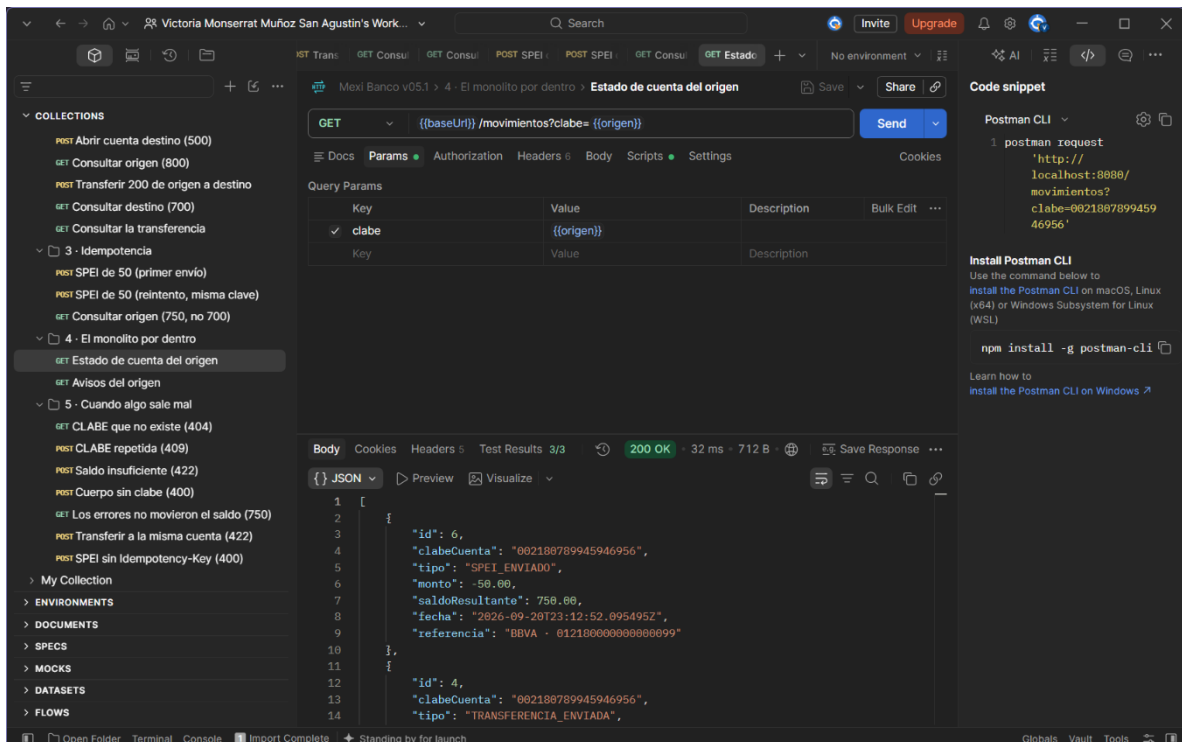
3.3. Consultar origen (750, no 700): `{{baseUrl}}/cuentas/{{origen}}`



`GET` es la consulta de la cuenta de origen tras la operación SPEI, la cual responde el saldo de la titular "Ana" se encuentra actualizado en `750.00`, lo que refleja de forma correcta la disminución acumulada tras restar la transferencia interna previa (200) y la transferencia externa SPEI (50), validando así la correcta consistencia y actualización de los saldos en el sistema.

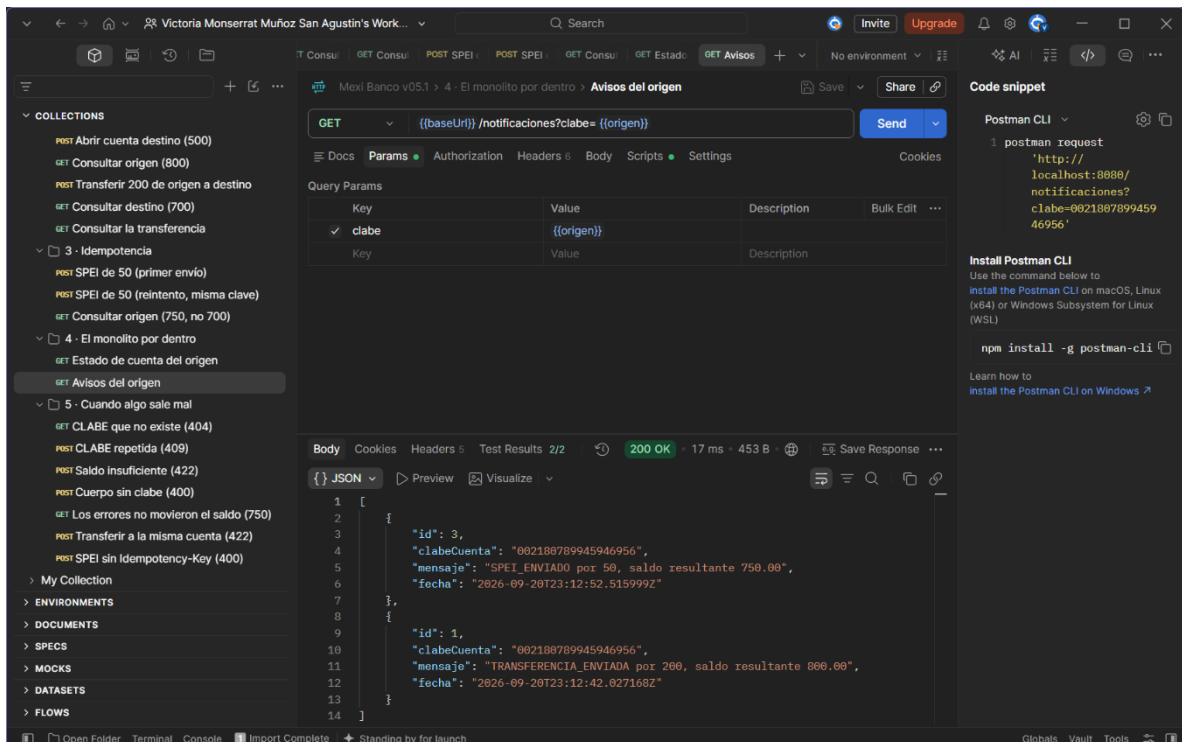
4. El monolito por dentro

4.1. Estado de cuenta del origen: `{{baseUrl}}/movimientos?clabe={{origen}}`



`GET` para el estado de cuenta del origen, donde se despliega una lista con el historial de movimientos de la cuenta.

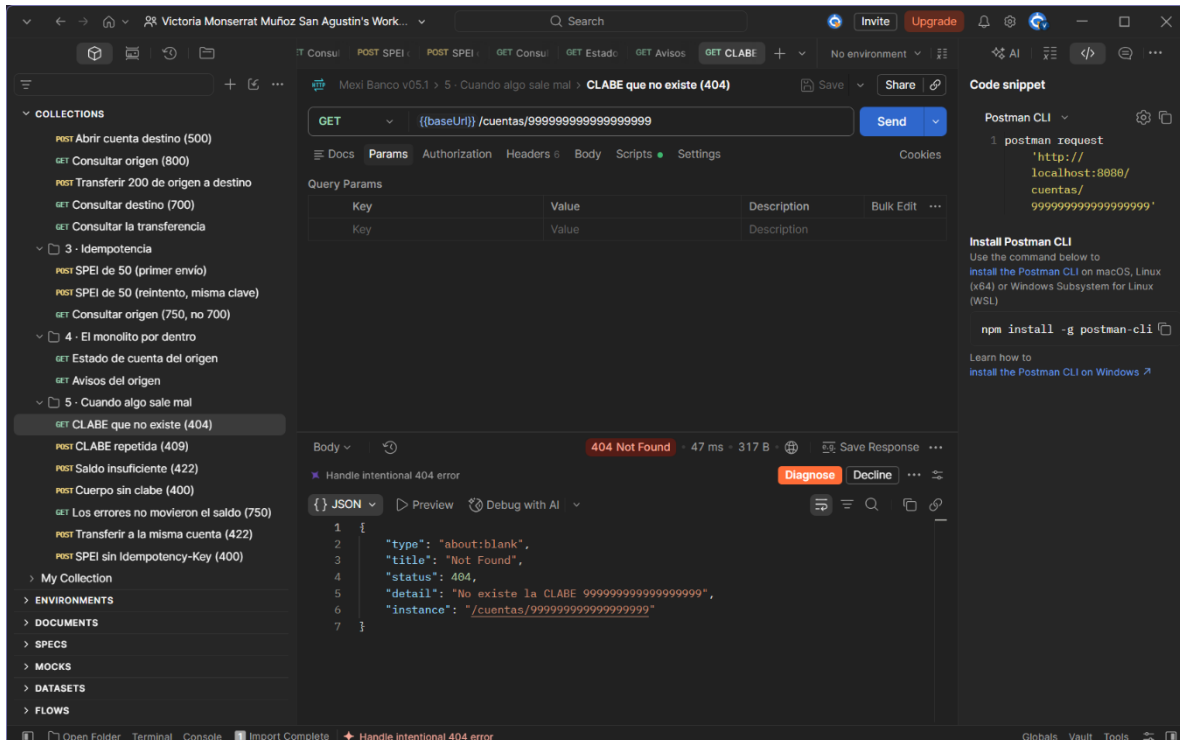
4.2. Avisos del origen: `{{baseUrl}}/notificaciones?clabe={{origen}}`



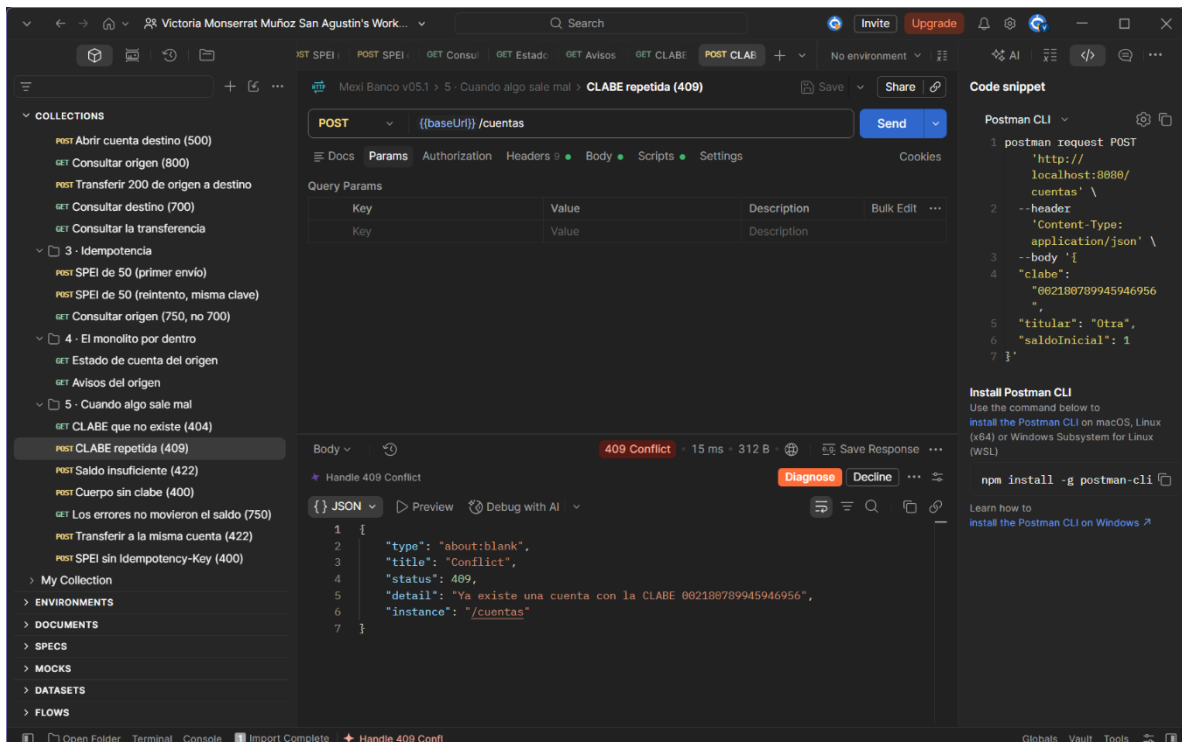
`GET` para los avisos de la cuenta de origen, donde la respuesta se despliega una lista con las notificaciones registradas para la cuenta, mostrando los mensajes detallados sobre las operaciones realizadas como el envío por SPEI y la transferencia interna, incluyendo los montos y los saldos resultantes correspondientes.

5. Cuando algo sale mal (observaciones resaltadas en amarillo)

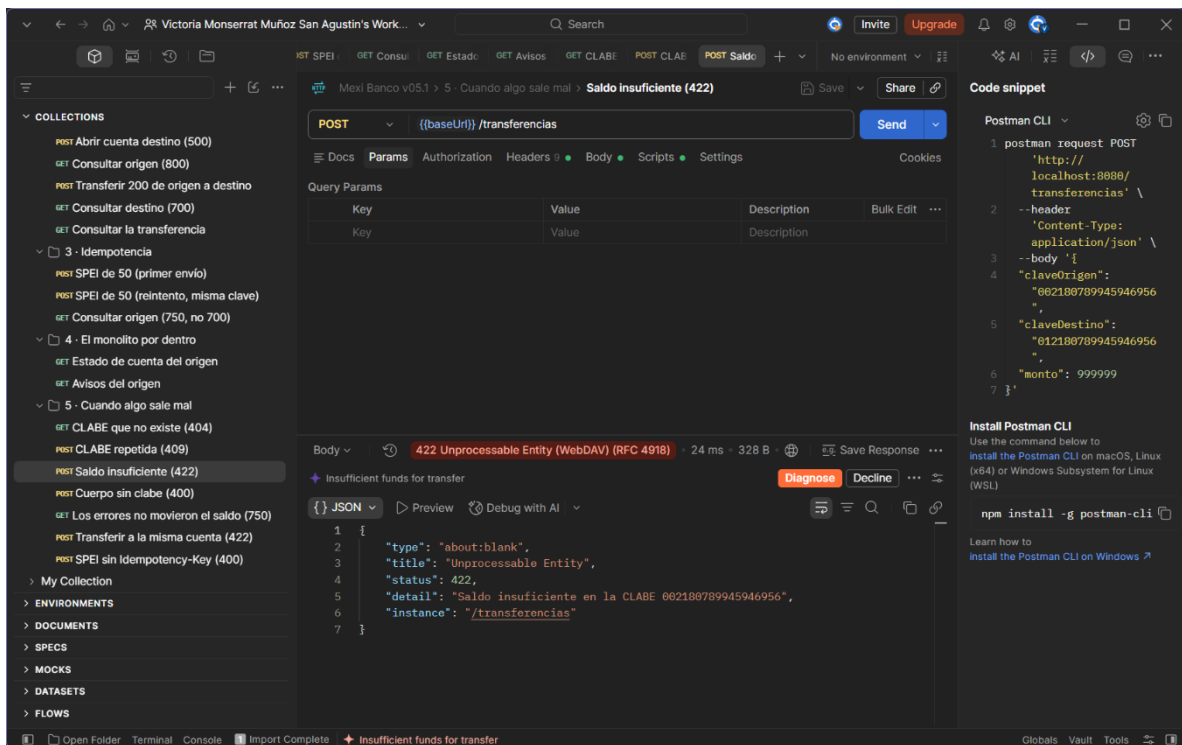
5.1. CLABE que no existe (404): `{{baseUrl}}/cuentas/999999999999999999`



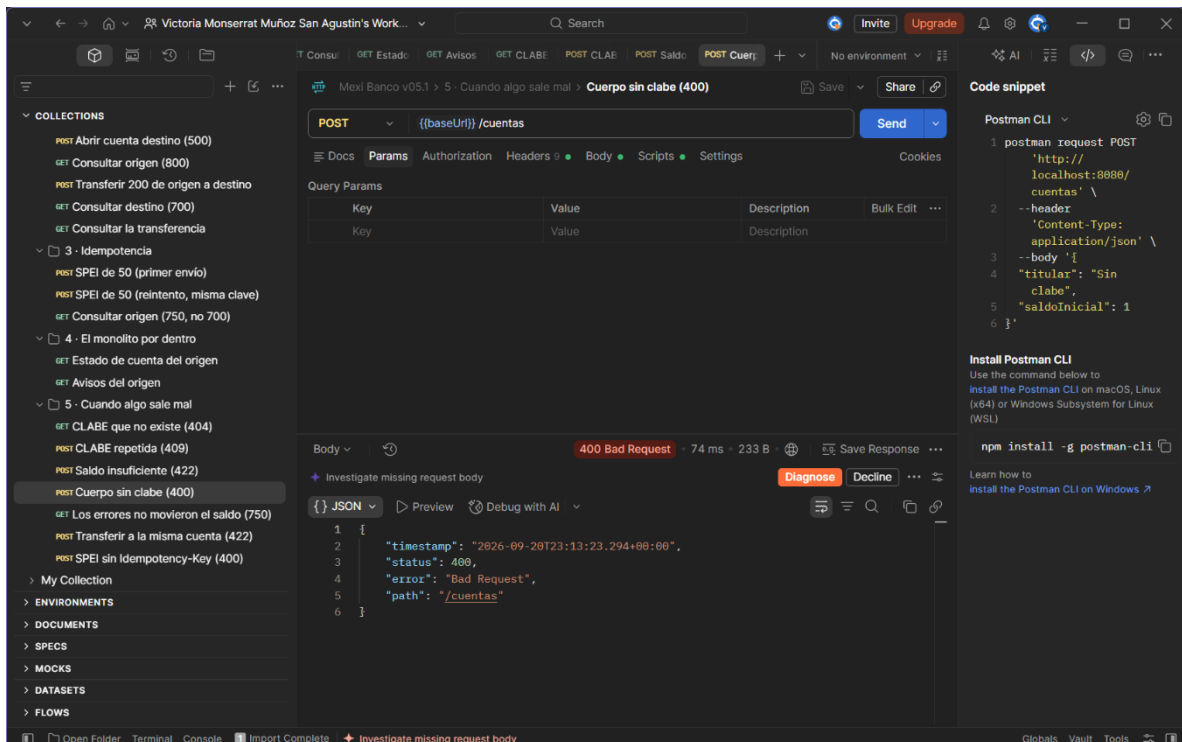
5.2. CLABE repetida (409): `{{baseUrl}}/cuentas`



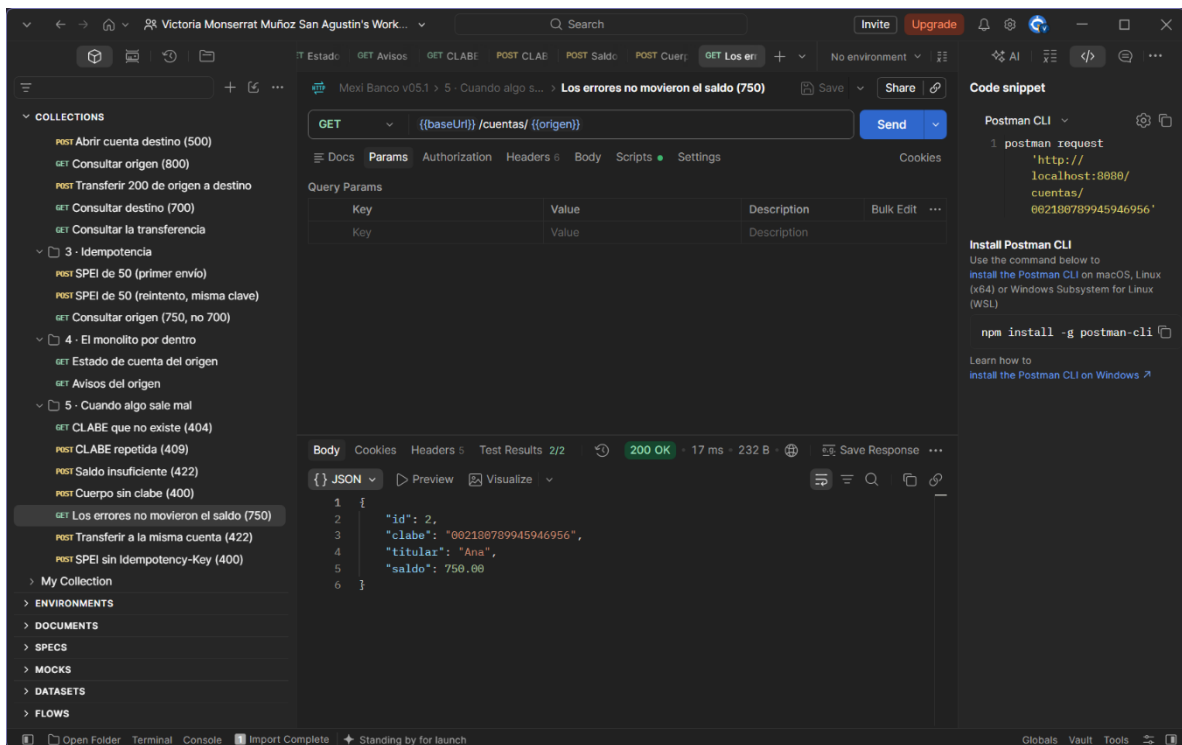
5.3. Saldo insuficiente (422): {{baseUrl}}/transferencias



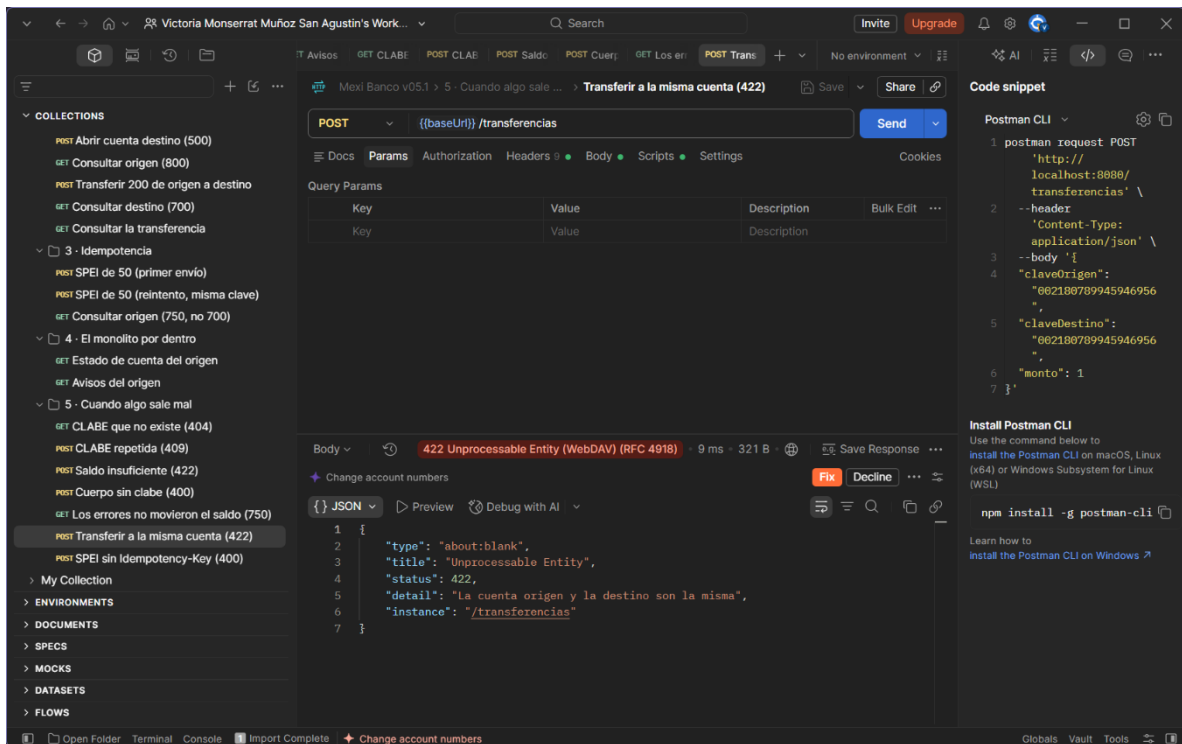
5.4. Cuerpo sin clave (400): {{baseUrl}}/cuentas



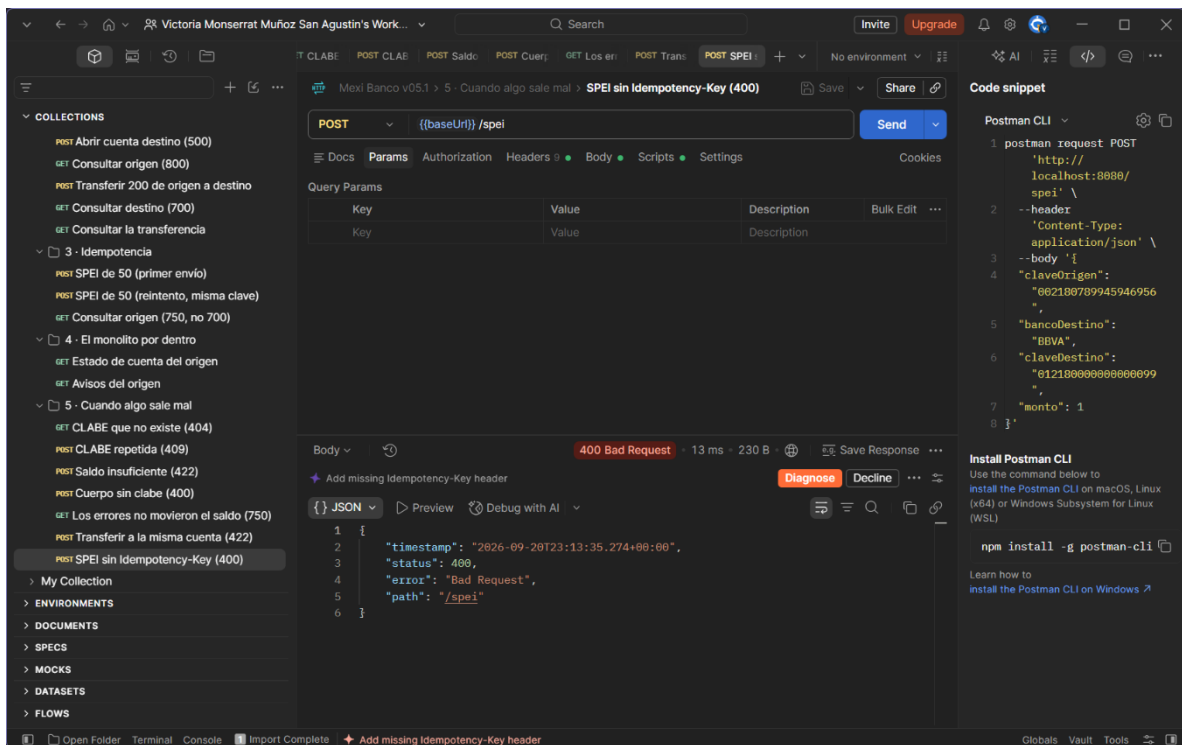
5.5. Los errores no movieron el saldo (750): {{baseUrl}}/cuentas/{{origen}}



5.6. Transferir a la misma cuenta (422): {{baseUrl}}/transferencias



5.7. SPEI sin idempotency-Key (400): {{baseUrl}}/spei



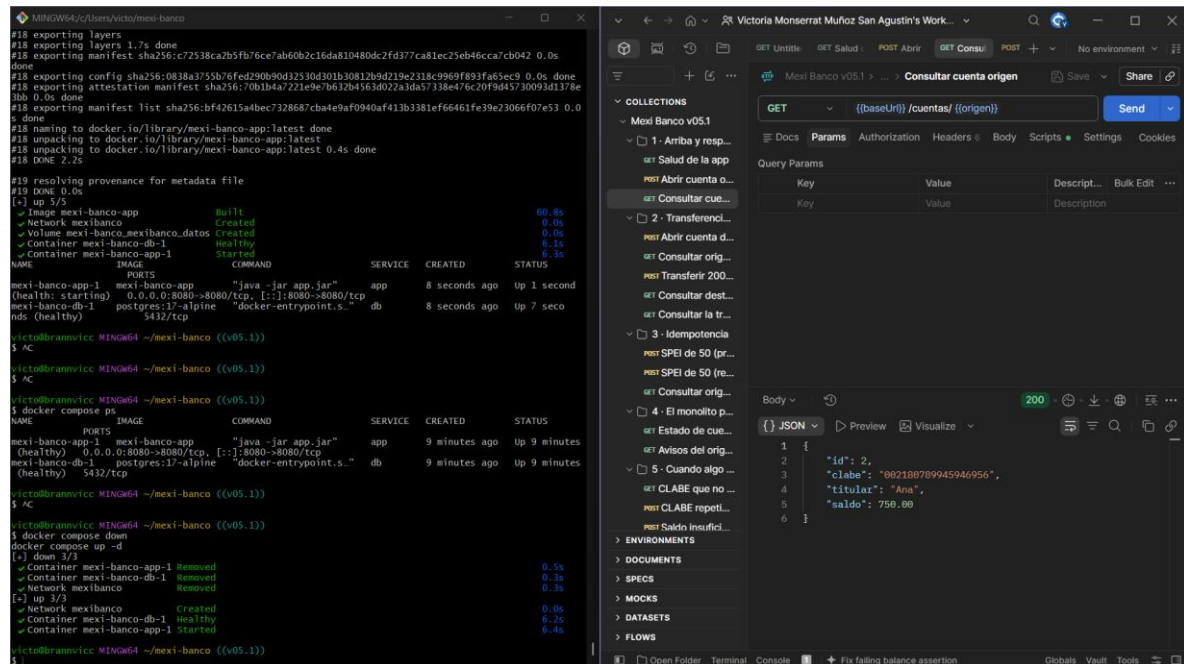
Paso 4: Bájalo con y sin volumen

Esto es lo que pasa cuando se hace lo siguiente:

apaga, vuelve a levantar y consulta tu cuenta en Postman

docker compose down

docker compose up -d



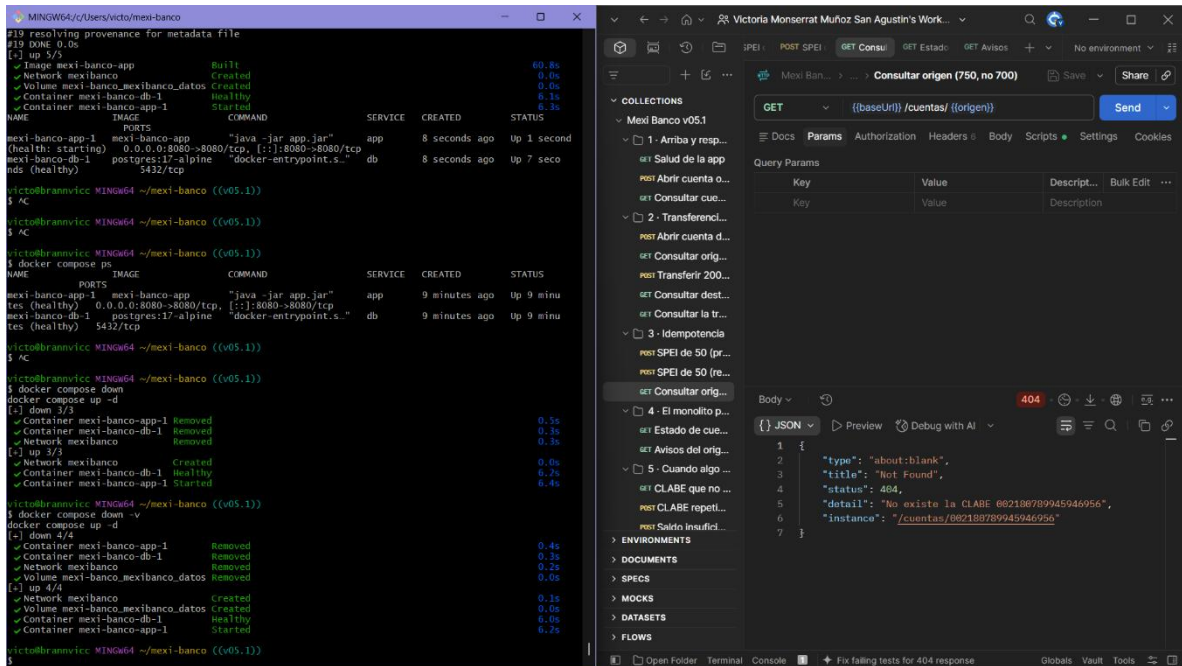
Qué observar: Al ejecutar el comando `docker compose down`, los contenedores se detienen pero el volumen con la información se conserva intacto, lo que permite que al volver a iniciar el sistema las cuentas y movimientos sigan disponibles y las peticiones de consulta respondan con normalidad.

Esto es lo que pasa cuando se usa lo siguiente:

ahora apaga borrando el volumen, levanta y consulta otra vez

docker compose down -v

docker compose up -d



Que observar: Al utilizar la bandera de limpieza con `docker compose down -v`, el volumen persistente se destruye por completo, provocando que al reiniciar la infraestructura la base de datos se inicialice totalmente vacía y cualquier intento de consultar registros previos arroje un error 404 Not Found, evidenciando así la diferencia fundamental entre conservar o eliminar el estado de la aplicación.